Objetivos da aula:

- Compreender o papel dos índices na otimização de consultas em bancos de dados relacionais;
- Criar e remover índices simples e compostos no PostgreSQL;
- Avaliar o impacto dos índices em consultas com filtros, intervalos e JOINs;
- Compreender a influência da seletividade e da ordem das colunas em índices compostos;
- Interpretar estratégias do otimizador, como Seq Scan, Index Scan, Bitmap Scan, Nested Loop e Hash Join;
- Tomar decisões fundamentadas sobre estratégias de indexação com base em dados reais.

Índice:

i. Índices em Bancos de Dados Relacionais
ii. Criação de Índices e Análise de Planos de Execução
iii. Tipos de Índices: B-tree, Hash e GiST

i. Índices em Bancos de Dados Relacionais

1.1. Introdução

À medida que o volume de dados armazenados em um banco cresce, o tempo de resposta das consultas passa a ser um fator crítico. Consultas que funcionam adequadamente em tabelas pequenas podem se tornar lentas e inviáveis em cenários reais de produção.

Os índices são estruturas auxiliares criadas com o objetivo de acelerar a recuperação de dados, reduzindo o custo de acesso às tabelas. Eles são um dos principais mecanismos utilizados pelos SGBDs para otimização de consultas.

1.2. Motivação para o Uso de Índices

Sem a utilização de índices, o SGBD normalmente precisa realizar uma varredura completa da tabela (*sequential scan*) para localizar os registros que satisfazem uma condição.

Exemplo conceitual:

```
SELECT *
FROM pessoa
WHERE id_pessoa = 1500;
```

Se a tabela aluno contiver milhares ou milhões de registros, a busca sem índice exige a leitura de todas as linhas até encontrar o valor desejado.

Os índices surgem para reduzir drasticamente esse custo, permitindo que o SGBD localize os registros de interesse de forma mais eficiente.

1.3. O Que é um Índice

Um índice é uma estrutura de dados auxiliar, associada a uma ou mais colunas de uma tabela, que mantém os valores dessas colunas organizados de forma a permitir buscas rápidas.

Características importantes:

- O índice **não substitui** a tabela;
- Ele armazena referências (ponteiros) para as linhas da tabela;
- O SGBD decide automaticamente se um índice será utilizado ou não;
- O uso de índices envolve trade-offs entre leitura e escrita.

Analogia comum:

> Um índice em banco de dados é semelhante ao índice de um livro: ele não contém todo o conteúdo, mas indica rapidamente onde encontrar a informação.

1.4. Índices e Desempenho

1.4.1 Consultas de Leitura

Índices são especialmente eficazes em consultas que utilizam:

- WHERE
- JOIN
- ORDER BY
- GROUP BY

Exemplo típico:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Souza';
```

Se existir um índice sobre a coluna `nome`, o SGBD pode evitar a varredura completa da tabela.

O resultado mostra que foram percorridas todas as linhas da tabela `pessoa`:

```
"Seq Scan on pessoa ..."
"  Rows Removed by Filter: 99994"
"Execution Time: 6.717 ms"
```

1.4.2 Operações de Escrita

Por outro lado, índices impactam negativamente operações de escrita, pois:

- Cada INSERT exige atualização dos índices;
- Cada UPDATE pode exigir reordenação do índice;
- Cada DELETE exige remoção de entradas do índice.

Portanto, índices devem ser criados de forma criteriosa, considerando o perfil de uso do sistema.

1.5. Funcionamento Geral dos Índices

Internamente, o SGBD utiliza estruturas de dados específicas para implementar índices. No PostgreSQL, o tipo mais comum é baseado em árvores balanceadas (B-tree).

De forma simplificada:

- Os valores indexados são mantidos em **ordem**;
- A busca ocorre em **tempo logarítmico**, e não linear;
- O número de acessos a disco é significativamente reduzido.

1.6. Índices Automáticos e Índices Explícitos

1.6.1 Índices Criados Automaticamente

No PostgreSQL, alguns índices são criados automaticamente:

- Índice para chave primária (PRIMARY KEY)
- Índice para restrição UNIQUE

Exemplo:

```
id_pessoa INTEGER PRIMARY KEY
```

Nesse caso, o SGBD cria automaticamente um índice sobre `id_pessoa`.

1.6.2 Índices Criados Manualmente

Outros índices devem ser criados explicitamente pelo desenvolvedor, conforme a necessidade.

```
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```

A decisão de criar esse índice deve levar em conta:

- Frequência de consultas pela coluna;
- Cardinalidade dos dados;
- Volume da tabela.

1.7. Quando Criar um Índice

Boas práticas gerais:

- Criar índices em colunas:
  - usadas frequentemente em cláusulas WHERE;
  - utilizadas em junções (JOIN);
  - utilizadas em ordenações (ORDER BY);
- Evitar índices em colunas:
  - com poucos valores distintos;
  - raramente utilizadas em consultas;

- que sofrem muitas atualizações.

Índices não são uma solução universal; eles devem ser justificados pelo uso real.

1.8. Considerações Importantes

- Nem toda consulta se beneficia de índices;
- O otimizador de consultas pode **optar por não usar um índice**, mesmo que ele exista;
- Índices ocupam espaço em disco;
- O uso excessivo de índices pode degradar o desempenho geral do sistema.

Esses pontos reforçam que índices fazem parte de uma estratégia de otimização, não de uma regra absoluta.

---

ii. Criação de Índices e Análise de Planos de Execução

2.1. Comando CREATE INDEX

O comando CREATE INDEX é utilizado para criar um índice explícito sobre uma ou mais colunas de uma tabela.

Estrutura Geral

```
CREATE INDEX nome_indice
ON nome_tabela (coluna1, coluna2, ...);
```

Exemplo simples:

```
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```

Nesse exemplo:

- O índice é criado sobre a coluna `nome`;
- O SGBD poderá utilizá-lo em consultas que filtrem ou ordenem por essa coluna.

2.2. Índices Simples e Índices Compostos

2.2.1 Índice Simples

Criado sobre uma única coluna:

```
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```

Indicado quando:

- A coluna é frequentemente utilizada em WHERE;
- A consulta filtra por apenas um critério.

2.2.2 Índice Composto

Criado sobre mais de uma coluna:

```
CREATE INDEX idx_pessoa_nome_idade
```

```
ON pessoa (nome, idade);
```

Características importantes:

- A ordem das colunas **é relevante**;
- O índice pode ser usado para consultas que envolvam:
  - apenas `nome`, ou
  - `nome` e `idade` conjuntamente;
- Não é eficaz para consultas que utilizem apenas `idade`.

2.3. Comando DROP INDEX

O comando DROP INDEX remove um índice existente.

```
DROP INDEX idx_pessoa_nome;
```

Observações:

- A remoção de um índice não afeta os dados da tabela;
- Pode impactar negativamente o desempenho de consultas que dependiam dele;
- É uma operação comum durante ajustes de desempenho.

2.4. EXPLAIN e EXPLAIN ANALYZE

Para analisar como o SGBD executa uma consulta, utiliza-se o comando EXPLAIN.

```
EXPLAIN
SELECT *
FROM pessoa
WHERE nome = 'Ana Souza';
```

O EXPLAIN mostra:

- O plano de execução escolhido;
- Estimativas de custo;
- Estratégias de acesso às tabelas.

O comando EXPLAIN ANALYZE executa a consulta e exibe dados reais de execução:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Souza';
```

2.5. Análise de Plano sem Índice

Considere a consulta:

```
EXPLAIN ANALYZE
SELECT *
```

```
FROM pessoa
WHERE nome = 'Ana Souza';
```

Resultado:

```
"Seq Scan on pessoa (cost=0.00..1915.00 rows=6 width=23) (actual time=0.845..4.670 rows=6 loops=1)"
"  Filter: ((nome)::text = 'Ana Souza'::text)"
"  Rows Removed by Filter: 99994"
"Planning Time: 0.067 ms"
"Execution Time: 4.683 ms"
```

Na ausência de índice sobre `nome`, o plano geralmente indica:

- `Seq Scan` (varredura sequencial);
- Linhas selecionadas (`rows=6`);
- Linhas percorridas e removidas pelo filtro (`99994`). Somando com as linhas selecionadas, comprova-se que foram percorridas todas as linhas da tabela `pessoa`.
- Tempo de execução (`4.683 ms`).

2.6. Análise de Plano com Índice

Após a criação do índice:

```
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```

Executando novamente:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Souza';
```

Resultado:

```
"Bitmap Heap Scan on pessoa (cost=4.34..26.70 rows=6 width=23) (actual time=0.029..0.033 rows=6 loops=1)"
"  Recheck Cond: ((nome)::text = 'Ana Souza'::text)"
"  Heap Blocks: exact=6"
"  ->  Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.026..0.026 rows=6 loops=1)"
"        Index Cond: ((nome)::text = 'Ana Souza'::text)"
"Planning Time: 0.963 ms"
"Execution Time: 0.050 ms"
```

O plano tende a apresentar:

- `Index Scan` ou `Bitmap Index Scan`;
- Menor tempo real de execução (`0.050 ms`);

Isso demonstra, na prática, o impacto positivo do índice.

2.7. Quando o Índice Não é Utilizado

Mesmo existindo um índice, o otimizador pode optar por **não utilizá-lo**. Exemplos comuns:

- A tabela é muito pequena;
- A consulta retorna grande parte das linhas. Assim como aconteceu na consulta que filtrou `nascimento > DATE '1970-01-01'`.
- A coluna possui baixa seletividade;
- A condição não é compatível com o índice.

Exemplo:

```
CREATE INDEX idx_pessoa_nascimento
ON pessoa (nascimento);
```

`EXPLAIN ANALYZE`
`SELECT *`
`FROM pessoa`
`WHERE nascimento > DATE '1990-01-01';`
	Utilizou o índice:
	`Bitmap Heap Scan on pessoa (cost=411.37..1436.17 rows=28784 width=23) (actual time=0.968..2.840 rows=28451 loops=1)`

`EXPLAIN ANALYZE`
`SELECT *`
`FROM pessoa`
`WHERE nascimento > DATE '1970-01-01';`
	Não utilizou o índice:
	`Seq Scan on pessoa (cost=0.00..1915.00 rows=56995 width=23) (actual time=0.011..5.907 rows=57034 loops=1)`

Se quase todos os registros satisfazem a condição, o `Seq Scan` pode ser mais eficiente.

2.8. Comparação Prática: Com Índice × Sem Índice

Uma estratégia didática recomendada:

1. Executar a consulta sem índice;
2. Analisar o plano (Seq Scan);
3. Criar o índice;
4. Executar novamente a consulta;
5. Comparar custo, tempo e estratégia.

Esse processo reforça a ideia de que **índices devem ser avaliados empiricamente**, não apenas teoricamente.

iii. Tipos de Índices: B-tree, Hash e GiST

3.1. Introdução

Nos capítulos anteriores, foram apresentados os conceitos gerais sobre índices e sua criação prática. No entanto, nem todos os índices são implementados da mesma forma nem atendem aos mesmos tipos de consultas.

Os SGBDs modernos oferecem diferentes tipos de índices, cada um baseado em estruturas de dados específicas e otimizado para determinados padrões de acesso aos dados.

Neste capítulo, serão estudados os principais tipos de índices disponíveis no PostgreSQL Global Development Group:

- B-tree
- Hash
- GiST (Generalized Search Tree)

O objetivo é compreender quando utilizar cada tipo, suas vantagens, limitações e impacto no desempenho das consultas.

3.2. Índice B-tree

3.2.1 Conceito

O índice B-tree (*Balanced Tree*) é o tipo de índice padrão no PostgreSQL e o mais amplamente utilizado em bancos de dados relacionais.

Ele é baseado em uma árvore balanceada, na qual:

- Os dados permanecem ordenados;
- A profundidade da árvore é mantida equilibrada;
- As operações de busca ocorrem em tempo aproximadamente logarítmico.

Quando nenhum tipo de índice é especificado, o PostgreSQL cria automaticamente um índice B-tree.

3.2.2 Criação de Índice B-tree

```
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```

Ou explicitamente:

```
CREATE INDEX idx_pessoa_nome
ON pessoa USING BTREE (nome);
```

3.2.3 Consultas Otimizadas por B-tree

Índices B-tree são utilizados em consultas que envolvem:

- Comparações de igualdade (=)
- Intervalos (<, >, <=, >=)
- BETWEEN

- ORDER BY
- GROUP BY
- JOIN

3.2.4 Limitações do B-tree

- Não é eficiente para buscas baseadas em padrões complexos;
- Não é adequado para dados espaciais ou estruturas não ordenáveis;
- Pode não ser utilizado quando a seletividade da coluna é baixa.

3.3. Índice Hash

3.3.1 Conceito

O índice Hash utiliza uma função de dispersão (hash) para mapear valores diretamente para posições no índice.

Diferentemente do B-tree:

- Os dados **não são mantidos ordenados**;
- O acesso é feito por igualdade exata;
- Não há suporte a buscas por intervalo.

3.3.2 Criação de Índice Hash

```
CREATE INDEX idx_carro_placa_hash
ON carro USING HASH (placa);
```

3.3.3 Consultas Otimizadas por Hash

Índices Hash são utilizados exclusivamente em consultas com **comparação por igualdade**:

```
SELECT *
FROM carro
WHERE placa = 'ABC1A34';
```

3.3.4 Limitações do Índice Hash

- Não suporta ORDER BY;
- Não suporta buscas por intervalo;
- Historicamente apresentava limitações de confiabilidade (corrigidas nas versões mais recentes do PostgreSQL);
- Em muitos casos, **não apresenta vantagens significativas sobre o B-tree**.

Por esse motivo, índices Hash são menos utilizados na prática.

3.4. Índice GiST (Generalized Search Tree)

3.4.1 Conceito

O índice GiST é um índice generalizado, projetado para suportar **tipos de dados e operadores complexos** que não se encaixam no modelo tradicional do B-tree.

Diferentemente de B-tree e Hash:

- O GiST **não impõe uma ordenação total**;
- Ele funciona como uma **infraestrutura para diferentes estratégias de busca**;
- Seu comportamento depende do tipo de dado indexado.

3.4.2 Exemplos de Uso do GiST

O GiST é utilizado principalmente em:

- Dados espaciais (coordenadas latitude e longitude);
- Intervalos de valores;
- Dados geométricos;
- Busca por similaridade.

Exemplo conceitual (intervalos):

```
CREATE INDEX idx_pessoa_nascimento_gist
ON pessoa
USING GIST (nascimento);
```

3.4.3 Limitações do GiST

- Maior complexidade conceitual;
- Custo maior de manutenção;
- Não é indicado para consultas simples por igualdade;
- Uso geralmente restrito a domínios específicos.

3.5. Critérios para Escolha do Tipo de Índice

Recomendações práticas:

- Utilize B-tree como padrão;
- Considere Hash apenas para consultas intensivas por igualdade muito específicas;
- Utilize GiST quando trabalhar com:
    - Intervalos;
    - Dados espaciais;
    - Operadores especializados;
- Sempre valide a escolha com EXPLAIN ANALYZE.